# Creating a mapping file with Starmap

Catherine Dalzell

2023-05-12

## Contents

**1 Using `starmap`** **1**
1.1 The `starmap formula` . . . . . 2
1.1.1 Starmap tokens . . . . . 2
1.1.2 Examples . . . . . 3
1.1.3 Illegal formulae . . . . . 3

**2 The Toronto Beaches data** **3**
2.1 The SMS Mappings . . . . . 4
2.1.1 Factors . . . . . 5
2.1.2 DateTimes . . . . . 6
2.2 The Ontology . . . . . 6

## 1 Using `starmap`

If external data is already in RDF format, the data format Stardog uses to build the graph, we can simply add the data to the database. On the command line, this is done using stardog data add. In `stardogR, we can use the data_add function. However, if the data are in some other format, such as SQL, CSV or JSON, we need to instruct Stardog how to build the graph. The Stardog Mapping Language, SMS is a simple scripting language that enables Stardog to convert tabular data into knowledge graph data (RDF). There is a tutorial and some examples for using SMS in the Stardog documentation. This vignette assumes that the reader already knows how to construct a mapping file for importing CSV files.`

`starmap allows the R user to build a simple SMS file for mapping data frames to Stardog. starmap makes a lot of decisions, which may not be suitable for every application. Nevertheless, starmap output can save time by setting up the boilerplate and building a framework. You can then edit the details suit your needs. starmap returns a list containing two strings. The first is the SMS mapping in string format, and the second is a simple ontology for the terms in the mapping.`

starmap requires four arguments.

```
library('stardogR')
args(starmap)
```

```
function (formulae, data, prefix = "", urn = "http://stardog.com/")
NULL
```

**formulae** a vector of R-formula style that describe the structure of the graph we want to build. See below.

**data** a data frame containing the data to be mapped. Note that starmap only uses the names of the data frame and the data types. One could submit the first few rows of a dataframe and get the same results. This can be useful when working with very large files.

**prefix** an abbreviated form of the URN that will be used to build node and property names

**urn** the URN (Universal Resource Name). In theory, it would be legitimate to have several prefixes and URN's within the same mapping file, but that flexibility is not offered in starmap. In that situation, one would need to edit the mapping or invoke the function for each URN.

## 1.1 The starmap formula

A starmap formula indicates how the data should relate in the knowledge graph. The syntax is based on the R formula language. Complex relationships can be built up using several formulae, passed to starmap as a vector. Here are the tokens used by starmap and their usage. Assume that the data have columns with names v1, v2, v3, d1, d2, d3, d4. Variables v1, v2, v3 will be class nodes, and d1, d2, d3, & d4 will be datatype properties.

### 1.1.1 Starmap tokens

~ The tilde relates two class nodes. "v1 ~v2" creates a directed relationship from v1 to v2, both of which have URN's.

| The pipe symbol relates an IRI node (class node with URN) to its literal (or datatype) properties. "v1 ~d1 + d2" creates datatype relationships between v1 and literals d1 and d2

# The expression #varname creates an IRI node from the row number and builds the URN from varname and the supplied prefix.

: The colon is used to pull in a sequence of literal properties from the data frame. 4:7 maps columns 4 through 7 inclusive as datatype properties. Equally, the columns names can be used: d1:d4. If

d1 is after d4, reading left to right across the data frame, an error is thrown.

### 1.1.2 Examples

**"v1"** Just maps column v1 as a class node.

**"v1 | d1 ~ v2 | d2 + d3"** connects v1 to v2, giving datatype d1 to v1, and d2, d3 to v2.

**c("v1 | d1 ~ v2 | d2 + d3", "v3 ~ v1")** as above, while also connecting v3 to v1.

### 1.1.3 Illegal formulae

**"v1 ~ v2 ~ v3"** There can only be one tilde per formula. Use instead, (c"v1 ~ v2", "v2 ~ v3")

**"d1 + d2"** Dataype properties need to be related to their subjects.

**"~ v2"** Not meaningful. To map just one column, do "v2"

**Other errors** starmap will throw an error if a term in the formula is not a column name of the database, or preceded by a # symbol.

## 2 The Toronto Beaches data

stardogR comes with a set of data drawn from the open data portal of the City of Toronto. This set contains daily readings of eColi levels at each of Toronto's public beaches over the summer months of 2007 through 2013.

```
data('beaches', package = 'stardogR')
names(beaches)
```

```
[1] "Beach_ID"       "Beach_Name"     "Sample_Date"    "Publish_Date"
[5] "eColi_Level"    "Beach_Status"   "Beach_Advisory"
```

Each row contains the results of one sample. I need an IRI node for each row. The sample date, eColi level and beach status are literal properties of the row. In addition, we might want to include column Beach_Advisory, which contains a text description of the results of the test. The beaches should be mapped to their own IRI nodes, with Beach_Name as a literal.

```
results <- starmap("#sample | Sample_Date + eColi_Level + Beach_Status +
                     Beach_Advisory ~ Beach_ID | Beach_Name",
                   beaches,
                   prefix = "beach",
                   urn = "http://beach.stardog.com/"
```

```
                  )
names(results)
```

```
[1] "sms"  "onto"
```

## 2.1 The SMS Mappings

```
cat(results$sms)
```

```
prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>
prefix xsd: <http://www.w3.org/2001/XMLSchema#>
prefix owl: <http://www.w3.org/2002/07/owl#>
prefix stardog: <tag:stardog:api:>
prefix beach: <http://beach.stardog.com/>

MAPPING
FROM CSV {
}
TO {
?sample_iri a beach:Sample ;
rdfs:label ?sample .
?sample_iri
beach:sampleDate ?Sample_Date_tr ;
beach:ecoliLevel ?eColi_Level_tr ;
beach:beachStatus ?Beach_Status_tr ;
beach:beachAdvisory ?Beach_Advisory_tr ;
 .
?Beach_ID_iri a beach:BeachId ;
rdfs:label ?Beach_ID .
?Beach_ID_iri
beach:beachName ?Beach_Name_tr ;
 .
?sample_iri beach:hasBeachId ?Beach_ID_iri .
}
WHERE {
BIND(TEMPLATE("http://beach.stardog.com/sample_{_ROW_NUMBER_}") as ?sample_iri)
BIND(xsd:date(?Sample_Date) as ?Sample_Date_tr)
BIND(xsd:integer(?eColi_Level) as ?eColi_Level_tr)
BIND(xsd:string(?Beach_Status) as ?Beach_Status_tr)
BIND(xsd:string(?Beach_Advisory) as ?Beach_Advisory_tr)
BIND(TEMPLATE("http://beach.stardog.com/Beach_ID_{Beach_ID}") as ?Beach_ID_iri)
BIND(xsd:string(?Beach_Name) as ?Beach_Name_tr)
}
```

Here is the generated mapping file. The prefix lines at the start

repeat the namespaces used by Stardog. The first five are namespaces defined by Stardog whenever a database is created. Teh ‘beach‘ prefix is, of course, the one supplied in the function call.

‘starmap‘ converts the dataframe to serialized CSV, requiring the ‘FROM CSV‘ clause. ‘starmap‘ could be used as a starting point for building mappings from SQL or JSON, by changing the ‘FROM‘ clause accordingly. See stardog documentation for details.

‘starmap‘ uses the names of the data frame to build properties and classes. Best results are obtained if the variable names are in snake_case, or camelCase.

* Classes are formed from the variable name in PascalCase (first letter capitalized). If the variable name has an underscore, ‘starmap‘ removes the underscore and capitalizes the components. * Datatype properties are formed from camelCase (first letter is a miniscule) * Object properties take the form ‘has‘+PascalCase of the endpoint of the edge. So the relationship from ‘Sample‘ to ‘Beach_ID‘ is expressed as ‘hasBeachId‘.

Stardog receives imported data as strings, and SMS assumes, by default, that everything is a string. To convert to a different data type, SMS needs to apply one of the ‘xsd‘ conversion functions to the incoming strings. ‘starmap‘ uses the datatypes of the data frame to figiure out which function to use. Let’s take a look at the datatypes of the beaches data.

```
unlist(lapply(beaches, class))
```

```
      Beach_ID     Beach_Name    Sample_Date   Publish_Date    eColi_Level
     "integer"    "character"         "Date"    "character"      "integer"
  Beach_Status Beach_Advisory
   "character"    "character"
```

‘Sample_Date‘ has class "Date", so in SMS, we run the conversion

BIND(xsd:date(?Sample_Date) as ?Sample_Date_tr)

This expression says, basically, "convert the data in column ‘Sample_Date‘ using function ‘xsd:date‘ and store the result in variable ‘?Sample_Date_tr."

Strictly speaking, it is not necessary to convert string datatypes, like the name of the beach, using ‘xsd:string‘, but it makes the process explicit and transparent. Every mapped variable should have an entry in the ‘WHERE‘ clause.

### 2.1.1 Factors

Factors are a datatype unique to R. They allow data to be stored with both numeric codes and their corresponding labels. R allows easy conversion from factors to either integer or character type, a convenient trick

in data analysis. There is no corresponding data structure in ‘xsd‘ or in Stardog. R factors are imported as character strings, using the labels rather than the numeric codes. If, for some reason, one wanted to retain both codes and labels, it would be necessary to save these as two different columns of the data frame. A suitable mapping would ensure that they remained connected in the database.

### 2.1.2 DateTimes

Datetimes need some preparation to work properly in ‘starmap‘ and Stardog generally. Stardog works with strings. When I pass an R object of class ‘POSIXct‘ to Stardog, it will take the string representation of that datetime, not a numeric epoch. This string is then converted to a dateTime object in Stardog by way of function ‘xsd:dateTime‘. The problem is that ‘xsd:dateTime‘ recognizes only one format. The strings need to be formatted as "%Y-%m-%dT%H:%M:%S". In most systems, the default datetime representation leaves a space between the date and the time, but RDF data requires a **T. If the data are formatted as R's default, with a space and no T, ‘starmap‘ will map the data as a datetime, and the virtual import will claim success, but the data will not in fact be imported.**

## 2.2 The Ontology

**‘starmap‘ creates a simple ontology for the new classes and properties defined for the previous mapping.**

```r
cat(results$onto)
```

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix stardog: <tag:stardog:api:> .
@prefix beach: <http://beach.stardog.com/> .

<http://beach.stardog.com/Sample> a owl:Class ;
rdfs:label 'Sample' .
<http://beach.stardog.com/sampleDate> a owl:DatatypeProperty ;
rdfs:label 'sampleDate' ;
rdfs:domain <http://beach.stardog.com/Sample> ;
rdfs:range xsd:date .
<http://beach.stardog.com/ecoliLevel> a owl:DatatypeProperty ;
rdfs:label 'ecoliLevel' ;
rdfs:domain <http://beach.stardog.com/Sample> ;
rdfs:range xsd:integer .
<http://beach.stardog.com/beachStatus> a owl:DatatypeProperty ;
```

```
rdfs:label 'beachStatus' ;
rdfs:domain <http://beach.stardog.com/Sample> ;
rdfs:range xsd:string .
<http://beach.stardog.com/beachAdvisory> a owl:DatatypeProperty ;
rdfs:label 'beachAdvisory' ;
rdfs:domain <http://beach.stardog.com/Sample> ;
rdfs:range xsd:string .
<http://beach.stardog.com/BeachId> a owl:Class ;
rdfs:label 'BeachId' .
<http://beach.stardog.com/beachName> a owl:DatatypeProperty ;
rdfs:label 'beachName' ;
rdfs:domain <http://beach.stardog.com/BeachId> ;
rdfs:range xsd:string .
<http://beach.stardog.com/:hasBeachId> a owl:ObjectProperty ;
   rdfs:label 'hasBeachId' ;
   rdfs:domain <http://beach.stardog.com/Sample> ;
 rdfs:range <http://beach.stardog.com/BeachId> .
```

**We can now use ‘stardogR‘ to create the database and load the data. Details on the use of ‘stardogR‘ are available in another vignette.**

```
sg <- Stardog("http://localhost:5820", "admin", "admin")
sg <- use_database(sg, "beach", reset = TRUE)
add_namespace(sg, uri = "http://beach.stardog.com/", prefix = "beach")
```

```
Response [http://localhost:5820/admin/databases/beach/options]
  Date: 2023-05-29 16:32
  Status: 200
  Content-Type: application/json
  Size: 80 B
```

```
add_dataframe(sg, beaches, mapping = results$sms)
```

```
Response [http://localhost:5820/admin/virtual_graphs/import]
  Date: 2023-05-29 16:32
  Status: 200
  Content-Type: <unknown>
<EMPTY BODY>
```

```
add_ttl(sg, ttl = results$onto, graph = "beach:onto")
```

***Data added successfully!***

```
query(sg)
```

```
      n
1 43385
```